PARMCO

CHECKPOINT 2 / SOFTWARE DOCUMENTATION

Technical documentation for the **six project source files**: Raspberry Pi keyboard motor control and IR RPM sensing, ARM assembly stepping/timing, C/BlueZ Bluetooth communications, and the SwiftUI iPhone application.

Course: University of Georgia | ECSE 4235 | Fall 2026
System: Raspberry Pi 4 / Raspbian 12 Bookworm + iOS app using Swift and CoreBluetooth
Scope: CP2 Bluetooth command exchange, keyboard-driven motor, and Test A IR RPM sensing
Prepared: 27 October 2026

Code repository: Group 1 - Final_Project / CP2 (commit 3f3f4e9)

1. Software overview and source inventory

The Raspberry Pi uses **two separate C executables for CP2**: one for Bluetooth text messaging and one for keyboard motor operation and IR RPM measurement. The iPhone app is a BLE central that sends commands and displays replies. The Bluetooth commands do **not** actuate the motor at this checkpoint.

Project filename	Language	Primary responsibility
main_cp2_ir.c	C	Motor keyboard interface, status display, GPIO input and pulse-based IR RPM
motorDriver.s	ARM32 assembly	H-bridge enable, phase patterns, motor stepping, generic-timer wait loop
parmco_ble.c	C	BlueZ GATT peripheral, advertising, command receive and ACK notification
PARMCOApp.swift	Swift	Application entry point and main scene
BluetoothManager.swift	Swift	CoreBluetooth central, GATT discovery, write / notify, connection state
ContentView.swift	Swift	User interface, controls, connection and message display

Architecture and data flow

Phone: ContentView.swift → BluetoothManager.swift → BLE command write → parmco_ble.c on Raspberry Pi → terminal print / GATT notification → app response display.

Motor: RP4 keyboard → main_cp2_ir.c → motorDriver.s → E4235 class GPIO library → 4N25 optocoupler / SN754410NE bridge / stepper motor. Separately, main_cp2_ir.c samples the GPIO17 IR signal and calculates RPM.

2. Library attribution: external dependencies

Class library credit

The **E4235 Raspberry Pi 4 class library** is supplied by the UGA ECSE 4235 course/instructor, **not authored by AI**. The class-library documentation is revision **1.1.2 (12 February 2026)**. Its revision history credits contributors from the 2024, 2025 and 2026 ECSE 4235 classes; the named contributors appear in Section 9.

Library / component	Attribution and mechanism	Why the attached code uses it
E4235.h / libE4235.a	UGA ECSE 4235 course-provided ARM assembly GPIO class library. E4235_Init initializes GPIO mapping; E4235_Select chooses input/output; E4235_Read samples inputs; E4235_Write drives outputs.	Course-mandated GPIO path. Main C reads GPIO17 / sets pin modes; ARM assembly calls E4235_Write for motor phases and enable.
BlueZ 5.66	Linux Bluetooth stack; external BlueZ project. Exposes org.bluez GATT and LE advertising managers on system D-Bus.	Pi registers a local GATT service and advertises its 128-bit service UUID to iOS.

Library / component	Attribution and mechanism	Why the attached code uses it
GLib / GIO 2.74.6	GNOME GLib/GIO D-Bus and main-loop libraries; GVariant marshals values, GDBus exports objects/calls BlueZ, GMainLoop dispatches callbacks.	Implements the C-side GATT service, D-Bus properties and advertisements without Python.
Foundation + Combine	Apple frameworks: Foundation provides NSObject, Data/UTF-8; Combine provides ObservableObject and @Published state publishing.	BLE manager stores observable connection/messages and converts text to/from bytes.
CoreBluetooth	Apple BLE framework; CBCentralManager discovers/links; CBPeripheral discovers services and accesses GATT characteristics.	iPhone central scans for Pi service, writes commands, subscribes to ACK notifications.
SwiftUI	Apple declarative UI framework; App/WindowGroup establish app scene; @State and @StateObject drive re-rendering.	Builds local CP2 controls, connection status and the last received reply.

2.1 Standard C and POSIX facilities by header

Header(s)	How used / why needed
errno.h, time.h	errno/EINTR + nanosleep for the 400 ms coil settle; clock_gettime(CLOCK_MONOTONIC) timestamps IR edges.
poll.h, termios.h, unistd.h	Nonblocking single-key terminal polling; configure/restore terminal; read keys; test interactive terminal.
signal.h, stdlib.h	SIGINT/SIGTERM handler for requested exit; atexit cleanup (motor program); signals also used for BLE server shutdown.
stdbool.h, stdint.h	bool tracking of bridge/rotation and uint64_t pulse count.
stdio.h, string.h	Terminal status/error/ACK printing; strlen in the BLE server. The main motor source also uses stdio.h.
gio/gio.h, glib-unix.h	GIO D-Bus/GVariant APIs and GLib signal-to-main-loop hook (specific to parmco_ble).

3. main_cp2_ir.c - motor keyboard and IR sensing

3.1 Purpose and initialization

The C program uses E4235_Init(), configures GPIO26 as output and drives it low at startup, configures four motor-phase pins (18, 23, 24, 25) as outputs, configures GPIO17 as a digital input, applies an initial phase and calls motor_enable(0). It requires an interactive terminal and installs signal/atexit cleanup handlers. This enforces a software quiet start; the hardware circuit must independently guarantee safe boot.

Setting / source constant	Actual uploaded value	Consequence
STEP_MODE / STEPS_PER_REV	1 / 400	Half-step lookup table and commanded 400 phase steps per nominal revolution.

Setting / source constant	Actual uploaded value	Consequence
Initial rate	66 steps/s (integer 10 × 400 / 60)	Initial commanded speed = 9.90 RPM, not exactly 10.00.
MIN_RATE / MAX_RATE	30 / 3000 steps/s	Software rate limits; the physical motor remains subject to step loss and load.
GPIO mapping	17 IR; 26 enable; 18/23/24/25 phases	GPIO numbers are BCM identifiers, not physical header pins.
IR polarity / pulses	Active LOW / 2 pulses per revolution	Counts one accepted active edge for each of the two propeller blades.

3.2 Keyboard command map

Key(s)	Code action
e / d	Energize the H-bridge; or disable bridge and clear movement and IR measurement state.
c / v	Select internally coded -1 direction (CW observed) or +1 (CCW observed); direction follows the team-validated wiring orientation.
o	Request 400 commanded half-steps and then stop/disable; the one-revolution accuracy is assessed in the motor demonstration.
r / x	Continuous stepping / stop and disable the bridge.
f or + / l or -	Adjust current step rate by rate/20 (minimum 1), capped by software rate bounds; print status.
s / h / q	Print status / key map / request clean exit.

3.3 Actual IR measurement algorithm

The loop calls E4235_Read(17) to sample a digital signal. Candidate input levels must remain stable for **4 ms** before the stable level is updated. Each transition to the configured active level produces a pulse candidate, ignoring gaps shorter than **55 ms**. Accepted pulse timestamps use the Linux **CLOCK_MONOTONIC** clock. The code retains up to the most recent **four** valid inter-pulse intervals in a ring buffer.

```
Measured IR RPM = 60 / (2 × mean accepted inter-pulse interval in seconds)
Commanded RPM = commanded steps/s × 60 / 400
```

The **2** is the configured IR_PULSES_PER_REV, not the independent reference-blade count. The computed reading is zero while **rotating == false**, before there are two accepted pulses, or after a gap longer than **4.5 s**. The program prints commanded steps/s, commanded RPM, IR RPM, pulse count, bridge state and rotation state in status(), roughly once per second while running.

3.4 Main program functions

Function	What it really does
ir_sample_at / ir_record_blade_at	Poll and debounce GPIO17; timestamp accepted active edges and update pulse intervals.
ir_measured_rpm_at / status	Compute pulse-based RPM; format current status output.
start_motor / handle_key	Apply initial coil phase, enable bridge, choose one-turn or run mode; dispatch keyboard actions.
disable_bridge / cleanup	Stop, de-energize H-bridge, clear IR data; restore terminal settings at exit.
main	Initialize GPIO and terminal; alternate keyboard checks, sensor sampling and assembly stepping.

Measurement behavior: GPIO17 is polled between steps rather than through a hardware interrupt. Optical pulses shorter than the polling interval can be missed. The 4 ms stable-level condition and recent-interval averaging determine which pulse edges contribute to displayed IR RPM. This explains the sensing approach and its timing characteristics without substituting commanded speed for measured speed.

Demonstration: The team confirms that the CP2 keyboard motor-control and IR Test A demonstrations operated as expected. Test A compares the displayed, pulse-derived IR RPM with commanded speed and an independent reference-blade count at approximately 30, 60 and 90 RPM. The numeric observations are part of the test record rather than hard-coded values in this application.

4. motorDriver.s - ARM32 H-bridge driver

The source uses GNU ARM assembly syntax with **.arch armv7-a**, exports **motor_enable**, **motor_apply_phase**, **motor_step** and **motor_wait_step**, and calls the instructor-provided **E4235_Write** routine rather than writing peripheral registers directly.

Routine	Inputs / outcome	How it operates
motor_enable	r0 on/off	Masks to 0 or 1 and writes GPIO26 through E4235_Write.
motor_apply_phase	r0 mode (0 full, nonzero half)	Indexes phase table and writes four coil bits to GPIO18, 23, 24, 25 through E4235_Write.
motor_step	r0 direction; r1 mode	Moves the phase index forward or backward, wraps at 4 or 8 entries and applies phase pattern.
motor_wait_step	r0 period in microseconds; r1 direction; r2 mode	Reads ARM generic timer frequency and counter, waits in a busy loop until computed deadline, then calls motor_step.

Phase patterns: full-step bytes = `5, 6, 10, 9`; half-step bytes = `5, 4, 6, 2, 10, 8, 9, 1`. The source declares .rodata, .bss and executable .text sections and a non-executable-stack note. The motor controller invokes motor_wait_step repeatedly; the physical shaft angle is checked independently against the reference protractor.

Timing method: The assembly reads the ARM generic timer registers, converts the timer frequency to ticks per microsecond, and busy-waits until the deadline before advancing one motor phase. Its blocking wait also determines how often the C loop can poll the IR input.

5. parmco_ble.c - C/BlueZ Bluetooth peripheral

5.1 Purpose and external APIs

This is a GLib/GIO program that connects to system D-Bus and exports BlueZ-compatible GattService1, two GattCharacteristic1 objects, LEAdvertisement1 and a D-Bus ObjectManager. It registers the GATT service first and then registers its advertisement, keeping the GLib main loop active.

GATT element	UUID / property	Usage
Service	E4235001-2026-4000-8000-000000000001	Pi advertises and iPhone scans for this primary service.
Command (write)	E4235001-2026-4000-8000-000000000002	Pi accepts phone UTF-8/ASCII text via WriteValue.
Response (notify)	E4235001-2026-4000-8000-000000000003	Pi exposes Value/Notifying and emits PropertiesChanged notifications.

5.2 Sequence and actual input handling

1. main() exports D-Bus objects and asynchronously calls GattManager1.RegisterApplication. 2. The registration callback calls LEAdvertisingManager1.RegisterAdvertisement. 3. The iPhone writes a string and newline to the command characteristic. 4. method_call() accepts 1-18 raw bytes, maps non-printing bytes to spaces, strips whitespace, prints `[PARMCO] RX: ...` and invokes send_ack(). 5. send_ack() prints `[PARMCO] TX: ACK:...` and emits a response Value change only if the notification subscription flag is set.

Named external interface	Why it is used
GDBusConnection / GDBusInterfaceVTable	Expose D-Bus properties and handle BlueZ method invocations in C.
GVariant / GVariantBuilder	Encode Bluetooth UUIDs, flags, byte arrays and structured D-Bus replies.
GMainLoop and g_unix_signal_add	Dispatch asynchronous events and stop on SIGINT/SIGTERM.
org.bluez.GattManager1	Register/unregister the local custom GATT service.
org.bluez.LEAdvertisingManager1	Register/unregister the BLE advertising data so a phone can find the service.

5.3 Protocol details and boundaries

Recognized by the phone UI: `FASTER, SLOWER, DIR:CW, DIR:CCW, START, STOP`. The C write handler does **not enforce this allow-list**; it accepts any non-empty, at-most-18-byte printable command after sanitizing. The Pi acknowledgement is `ACK:<received text>`. These commands are logged only: **this server never calls GPIO, IR functions, or motor routines**.

The advertisement uses the **local name PARMCO** (not PARMCO-RP4); iOS discovers the correct device by service UUID. The code prints a TX message before the BLE notification outcome is independently known; the app must display the ACK to confirm reception.

6. iPhone Swift app source

6.1 PARMCOApp.swift

Imports **SwiftUI**. The `@main` PARMCOApp conforms to App and returns a WindowGroup containing ContentView(). It sets the application entry point and creates the only app scene visible in this source. No Bluetooth logic or motor control is performed here.

6.2 BluetoothManager.swift

Imports **Foundation** for NSObject/Data/text handling, **Combine** for ObservableObject/@Published, and **CoreBluetooth** for all BLE central/client interactions. The manager is an NSObject ObservableObject whose published properties are status, isReady, lastSent and lastReceived.

Code path	Actual function
scanAndConnect	Requires powered-on Bluetooth; scans for the custom service UUID, not a device name.
didDiscover / didConnect	Stops scanning, connects to first matching peripheral, then discovers the service and characteristic pair.
didDiscoverCharacteristics / didUpdateNotificationState	Verifies write+notify properties, subscribes to notifications, sets isReady = true and connected status.
send	Appends newline to command, writes UTF-8 using .withResponse, immediately updates lastSent (not proof of successful delivery).
didUpdateValueFor	Decodes received bytes as UTF-8 and publishes lastReceived, which the SwiftUI screen displays.
disconnect / didDisconnectPeripheral	Cancels connection, clears characteristics and sets disconnected state; no special ACK:CONNECT or ACK:DISCONNECT is generated.

Swift UI state updates use the CoreBluetooth main delegate queue (`queue: nil`); this fits the published properties in this small foreground CP2 app. The file has callback status/error handling but no reconnect retry backoff or persistent BLE pairing feature.

6.3 ContentView.swift

Imports **SwiftUI**; `@StateObject` retains BluetoothManager for the view, while `@State` holds **local, simulated** desiredRPM = 20, clockwise = true, startRequested = false and lastAction = None. NavigationStack, ScrollView, VStack, Button and Text build the interface.

The speed UI clamps displayed desired RPM to the range 10-100 in increments of 5 but still sends the FASTER/SLOWER message when a button is pressed at the boundary. Direction and START/STOP toggles change the local label and send a text command. The last button action, last sent command and most recent Pi acknowledgment are displayed. **The displayed RPM is not IR-measured RPM**, and tapping START does not start the motor in CP2.

iOS project settings: The attached Xcode project (`PARMCO.xcodeproj/project.pbxproj`) includes `NSBluetoothAlwaysUsageDescription` with the value "PARMCO uses Bluetooth to communicate with the Raspberry Pi 4". It specifies iOS deployment target 27.0, Swift language setting 5.0, iPhone/iPad device families and automatic code signing. The Bluetooth use description enables the app to request system Bluetooth access.

7. Build settings, compiler options and configuration flags

7.1 Build and linking commands

```
BLE server (C / GLib / GIO):
gcc -Wall -Wextra -O2 -o parmco_ble parmco_ble.c $(pkg-config --cflags --libs gio-2.0 glib-2.0)
```

```
Motor control and IR sensing (C + ARM32):
gcc -O2 -Wall -Wextra -no-pie main_cp2_ir.c motorDriver.s -o parmco_cp2_ir -lE4235
```

Flag / option	Meaning and reason
-Wall / -Wextra	Enable additional GCC warnings, useful for checking generated C. They help identify questionable constructs during compilation.
-O2	Enable standard optimizations. Important to document because it can influence loop behavior/timing.
-o NAME	Set output executable name.
-no-pie	Request a non-position-independent executable for the RP4 C+ARM assembly build.
-lE4235	Link the class-provided GPIO static library (libE4235.a).
pkg-config --cflags --libs	Obtain the installed GIO/GLib header paths, compile flags and linker flags; not an application runtime flag.
.arch armv7-a / .arch_extension idiv	Source assembly architecture and division feature directives, not CLI flags.

7.2 Source configuration / state flags

File / symbol	Actual meaning
main_cp2_ir.c: STEP_MODE = 1	Half-step output; nominal 400 commanded steps/rev.
main_cp2_ir.c: IR_ACTIVE_LEVEL = 0	Count a transition to LOW; depends on actual sensor polarity.
main_cp2_ir.c: IR_PULSES_PER_REV = 2.0	Assumes two detected blade events per mechanical rotation.
main_cp2_ir.c: rotating / bridge_enabled	Track software run and bridge states; drive zero-RPM reporting and status.
parmco_ble.c: is_notifying	Gates transmission of ACK notification via response characteristic.
BluetoothManager.swift: isReady	True only after notification subscription was enabled and characteristics were found.

8. Code authorship and GPIO mapping change

The project software was AI-generated, with **one human-reported change** to accommodate the physical wiring: the 4N25 enable GPIO assignment was changed from **BCM GPIO4** to **BCM GPIO26**. The attached C and ARM source use GPIO26 for the enable signal; no other human modifications were reported.

Change	Original design	As implemented	Reason
4N25 enable GPIO	GPIO4	GPIO26	Match the actual optocoupler enable wire on the Raspberry Pi header.

Authorship / change flag

The code-flag convention for this change uses paired comments to identify the human intervention and retains the previous assignment in the change record. This flag refers to the GPIO mapping only; other code changes are not attributed to the student.

```
/* [HUMAN EDIT BEGIN - 4N25 enable GPIO mapping] */
/* Original AI assignment: BCM GPIO4 */
/* Updated assignment: BCM GPIO26 (physical 4N25 enable wire) */
/* [HUMAN EDIT END] */
```

In **main_cp2_ir.c**, the GPIO26 enable path appears in the pin initialization, low-on-boot safety write and bridge-control calls. In **motorDriver.s**, the `motor_enable` routine selects GPIO26 before calling `E4235_Write`. The final source logic therefore agrees with the revised wiring.

Authorship boundaries: The ECSE 4235 GPIO library is instructor/course-provided; the project-specific C, assembly and Swift source is AI-generated except for the GPIO mapping change described above. The original GPIO4 assignment is preserved here as the prior design value.

9. Libraries and technical references

Class library credit: UGA ECSE 4235 Raspberry Pi 4 class library, E4235.h and libE4235.a, revision documentation v1.1.2. The documentation credits the Spring 2024 class and Olivia Beattie (initial overview), Matthew Sherritt and Blake Hudson (2025 updates), and Lucian Bracy (2026 initialization update).

Apple frameworks: SwiftUI, Combine, Foundation and CoreBluetooth are Apple platform libraries. Framework reference: developer.apple.com/documentation/corebluetooth.

BlueZ: Linux BlueZ Bluetooth stack and its GATT and LEAdvertising D-Bus interfaces. API reference: bluez.readthedocs.io/en/latest/gatt-api/.

GLib / GIO: GNOME Foundation GLib/GIO (GDBus, GVariant, main loop and Unix signal sources). Reference: docs.gtk.org/gio/.

GCC and POSIX: GNU Compiler Collection, C standard and POSIX APIs for timing, terminal input and signal handling. Reference: gcc.gnu.org/onlinedocs/.

Project source: Group 1 CP2 repository, commit 3f3f4e9d. Source descriptions in this report use the six attached file snapshots and the provided iOS project settings.